

Unit-5

Application Layer and QoS

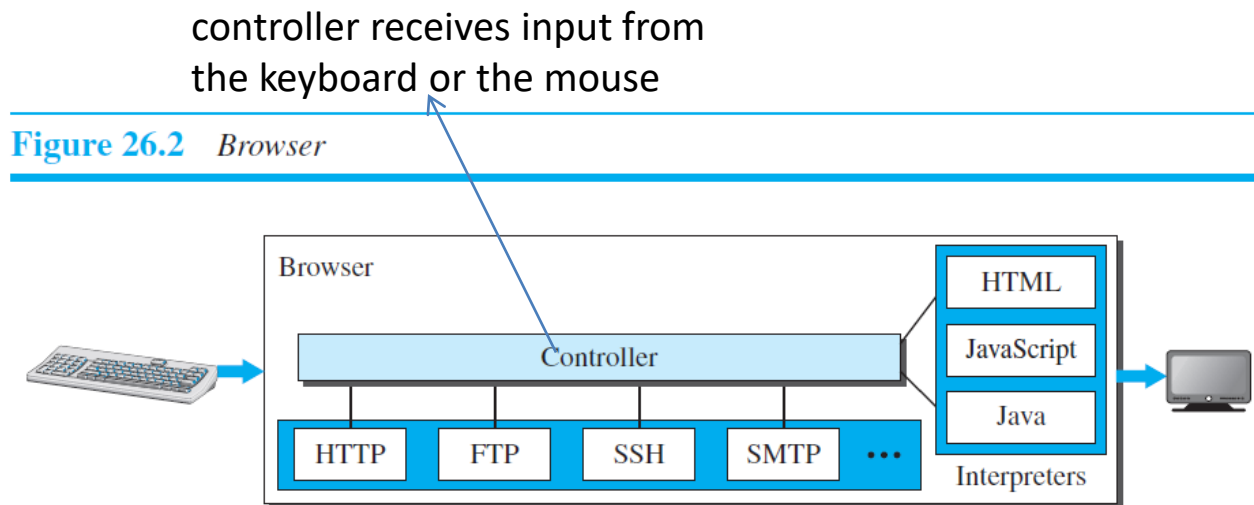
Dr.Pramod T.C
Assistant Professor,
Dept. of CSE
SIT, Tumkur
pramodtc@sit.ac.in

WORLD WIDE WEB AND HTTP

- The WWW today is a distributed client-server service, in which a client using a browser can access a service using a server.

Web Client (Browser) -Internet Explorer, Netscape Navigator, and Firefox

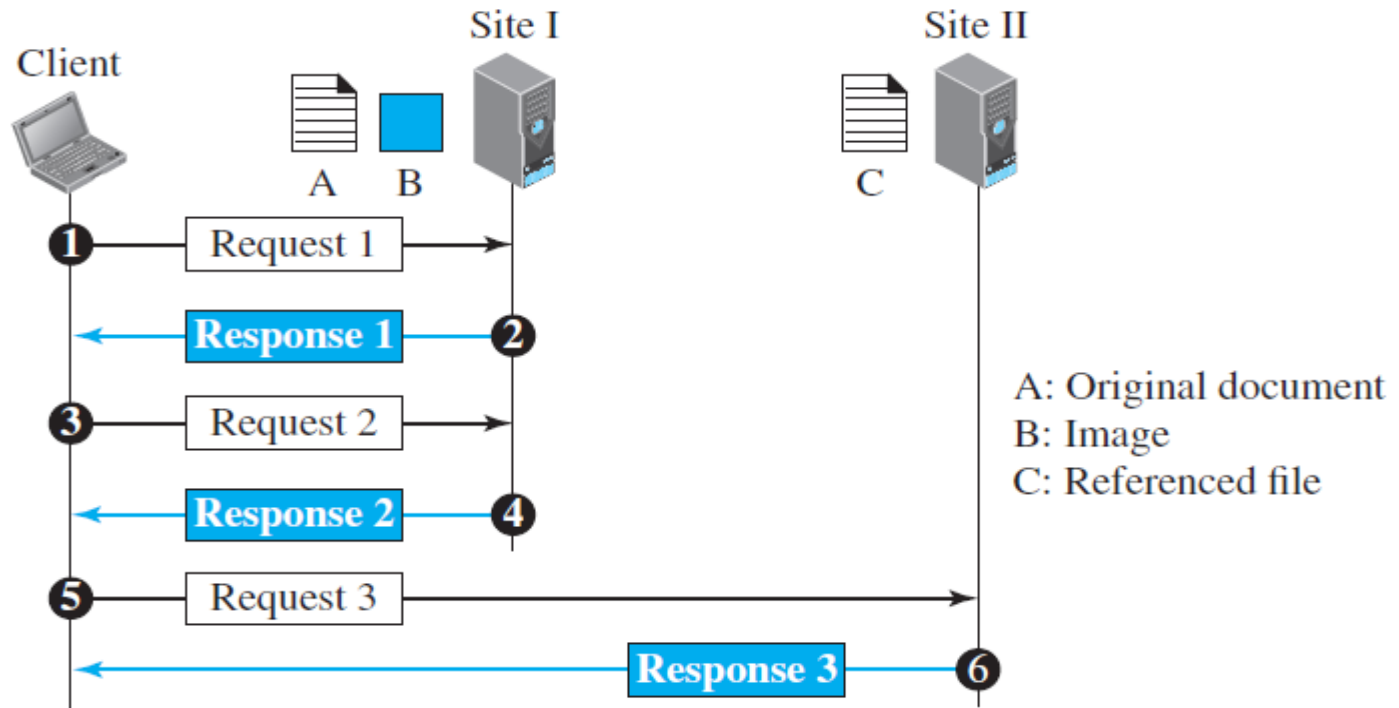
Each browser usually consists of three parts: a controller, client protocols, and interpreters



Web Server- Apache and Microsoft Internet Information Server

The web page is stored at the server. Each time a request arrives, the corresponding document is sent to the client.

Assume we need to retrieve a scientific document that contains one reference to another text file and one reference to a large image



The main document and the image are stored in two separate files (file A and file B) in the same site; the referenced text file (file C) is stored in another site

Uniform Resource Locator (URL)

To define a web page, we need three identifiers: *host*, *port*, and *path*

protocol://host/path

Used most of the time

protocol://host:port/path

Used when port number is needed

- ***Protocol***-HTTP, FTP etc
- ***Host***- IP address of the server or the unique name given to the server.
- ***Port***- 16-bit integer (HTTP protocol is used for accessing the web page-80)
- ***Path***- identifies the location and the name of the file in the underlying operating system (*/top/next/last/myfile*)

Web Documents

The documents in the WWW can be grouped into three broad categories: static, dynamic, and active.

Static Documents: are fixed-content documents that are created and stored in a server. The client can get a copy of the document only

- Static documents are prepared using one of several languages: HyperTextMarkup Language (HTML), Extensible Markup Language (XML), Extensible Style Language (XSL), and Extensible Hypertext Markup Language (XHTML)

Dynamic Documents

- When a request arrives, the web server runs an application program or a script that creates the dynamic document. The server returns the result of the program or script as a response to the browser that requested the document
- Example of a dynamic document is the retrieval of the time and date from a server. The client can ask the server to run a program such as the *date* program in UNIX and send the result of the program to the client
- to retrieve a dynamic document - *Common Gateway Interface (CGI)*, *Java Server Pages (JSP)*, *Active Server Pages (ASP)*, *ColdFusion*

Active Documents

- For many applications, we need a program or a script to be run at the client site. These are called **active documents**.
- **Ex: animation.** One way to create an active document is to use *Java applets*

HyperText Transfer Protocol (HTTP)

- To retrieve web pages from the Web (client-server)
- An HTTP client sends a request; an HTTP server returns a response. The server uses the port number 80; the client uses a temporary port number.
- HTTP uses the services of TCP

Nonpersistent versus Persistent Connections

- If the web pages, objects to be retrieved, are located on different servers, we do not have any other choice than to create a new TCP connection for retrieving each object
- if some of the objects are located on the same server, we have two choices: to retrieve each object using a new TCP connection or to make a TCP connection and retrieve them all.

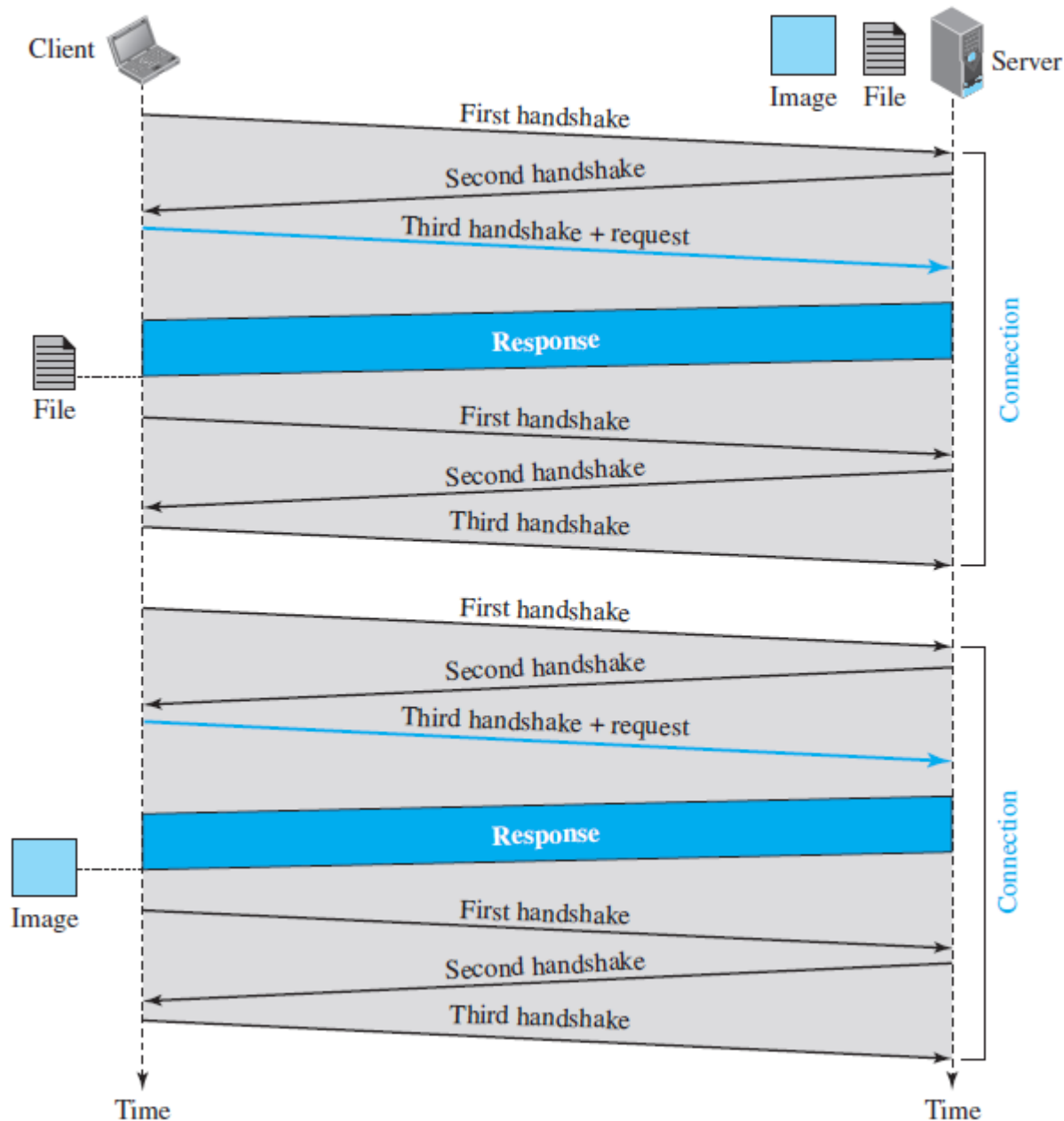
Nonpersistent Connections

In a **nonpersistent connection**, one TCP connection is made for each request/response

The following lists the steps in this strategy:

1. The client opens a TCP connection and sends a request.
 2. The server sends the response and closes the connection.
 3. The client reads the data until it encounters an end-of-file marker; it then closes the connection.
- if a file contains links to N different pictures in different files (all located on the same server), the connection must be opened and closed $N + 1$ times
 - The **nonpersistent strategy imposes high overhead on the server** because the server needs $N + 1$ different buffers each time a connection is opened.

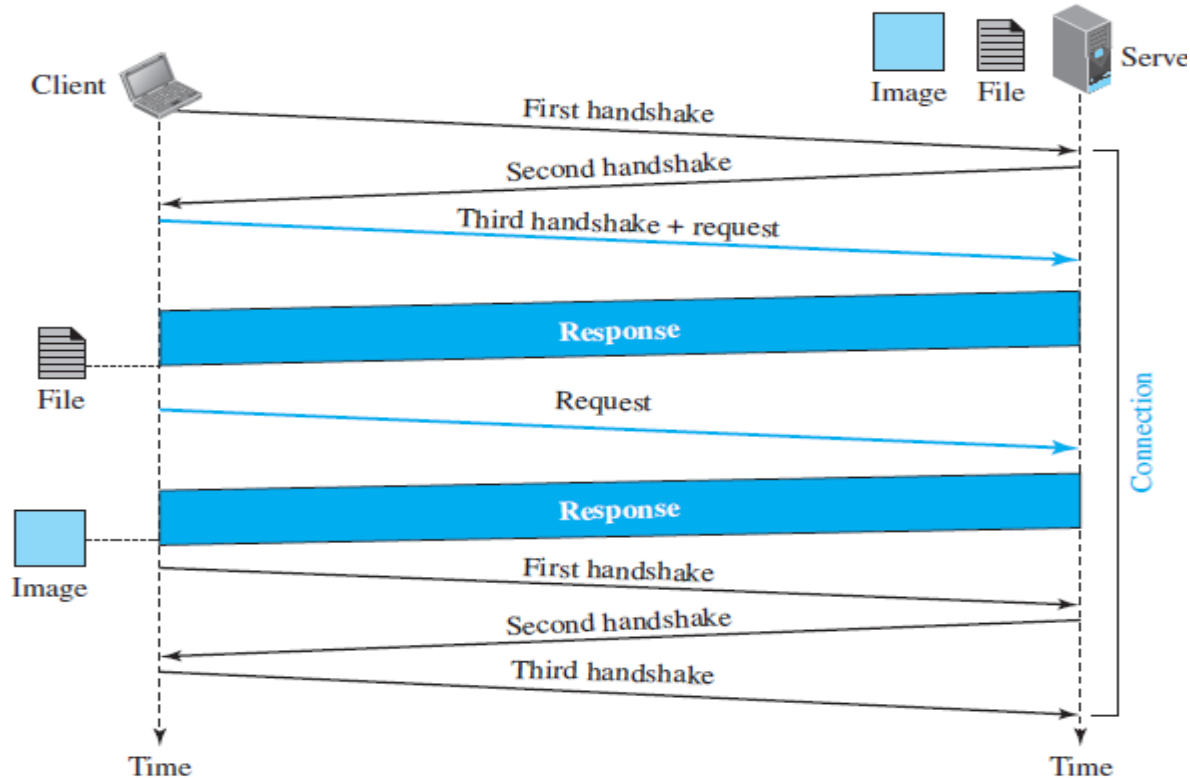
Nonpersistent Connections



The client needs to access a file that contains one link to an image. The text file and image are located on the same server. Here we need two connections.

Persistent Connections

- HTTP version 1.1 specifies a persistent connection by default.
- In a persistent connection, the server leaves the connection open for more requests after sending a response



- The server can close the connection at the request of a client or if a time-out has been reached.
- It imposes less overhead on the server

Non-persistent HTTP connection	Persistent HTTP connection
Non-persistent HTTP connection is one that is used for serving exactly one request and sending one response.	Persistent HTTP connection is one that can be used for serving multiple requests.
HTTP server closes the TCP connection automatically after sending a HTTP response.	HTTP server closes the TCP connection only when it is not used for a certain configurable amount of time.
A new separate TCP connection is used for each object.	A single TCP connection is used for sending multiple objects one after the other.
HTTP 1.0 supports non-persistent connections by default.	HTTP 1.1 supports persistent connections by default.
<u>Example-</u> Suppose a request has been made for a HTML page that contains 10 images (called objects). Then, With non-persistent connection, all the 11 objects (1 page + 10 images) will be sent one by one. For getting each object, a new separate connection will be opened and used.	<u>Example-</u> Suppose a request has been made for a HTML page that contains 10 images (called objects). Then, With persistent connection, all the 11 objects (1 page + 10 images) will be sent one after the other using a single TCP connection.

Message Formats

- The HTTP protocol defines the format of the request and response messages

Figure 26.5 *Formats of the request and response messages*

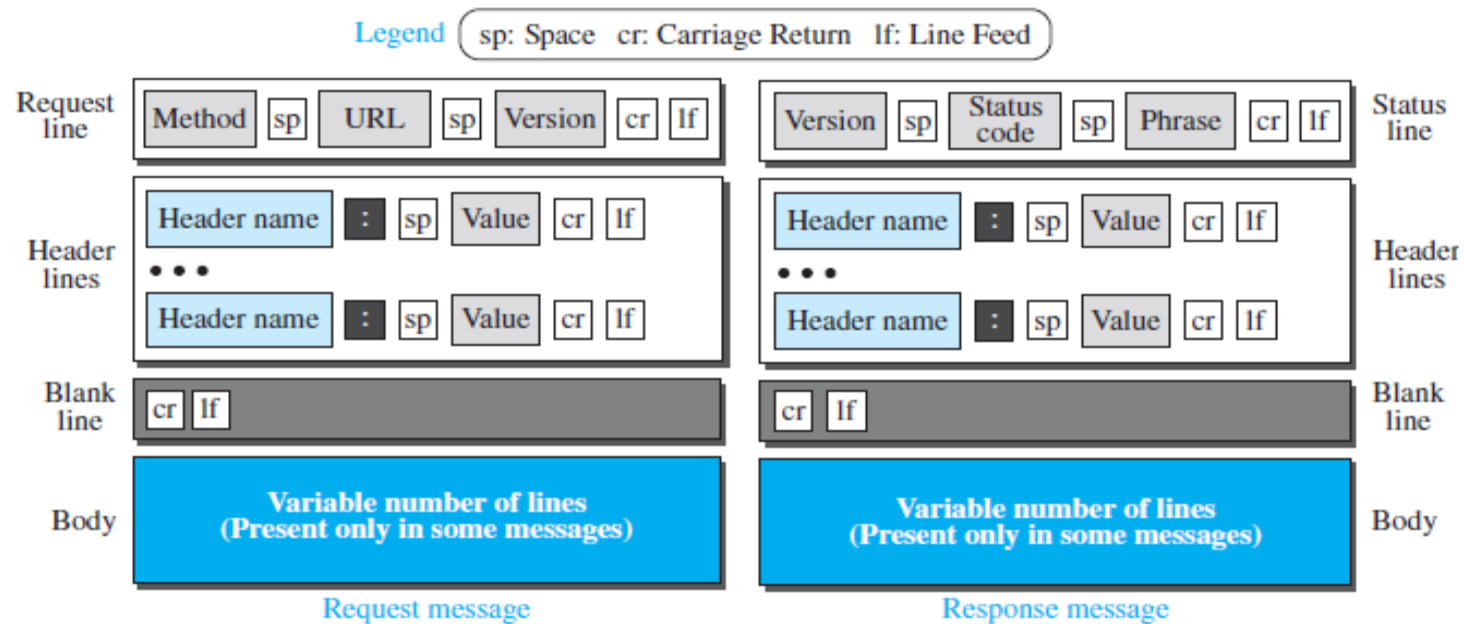


Table 26.1 *Methods*

<i>Method</i>	<i>Action</i>
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request (debugging)
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

Table 26.2 *Request header names*

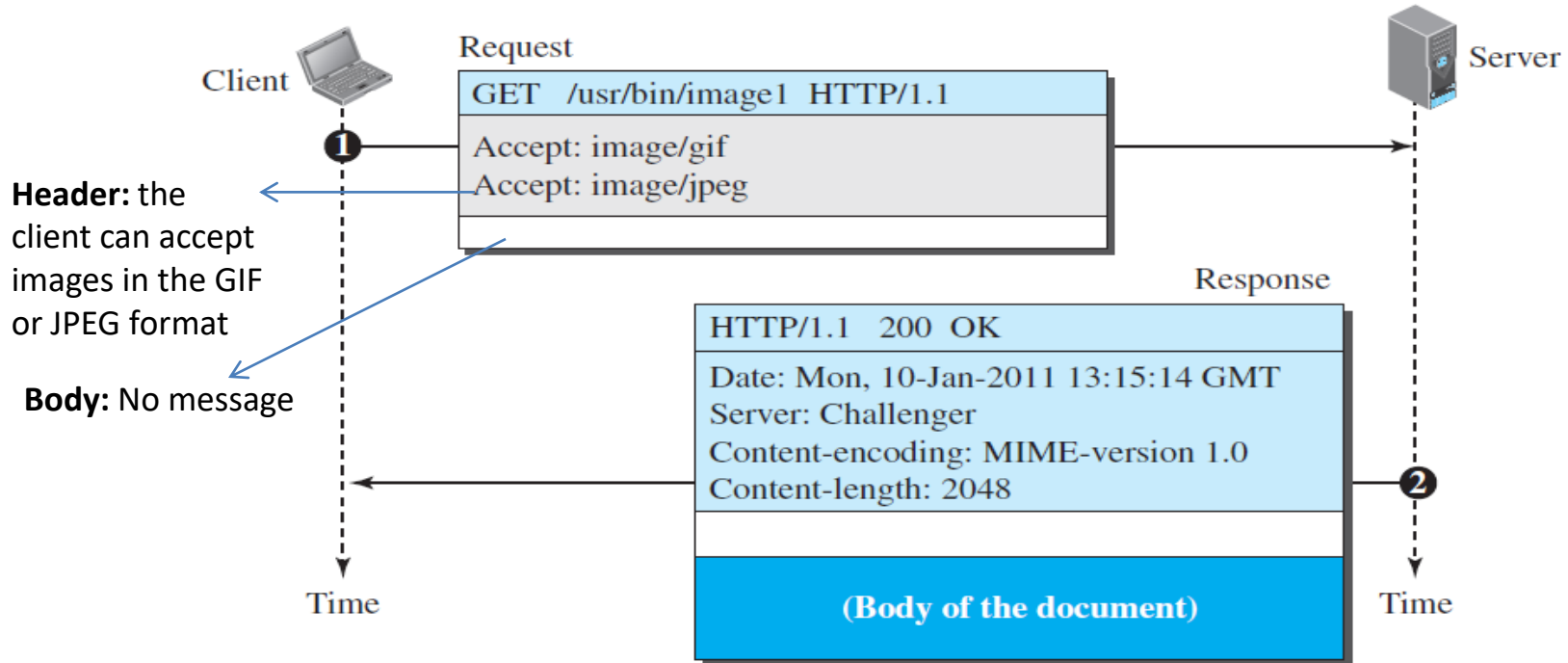
<i>Header</i>	<i>Description</i>
User-agent	Identifies the client program
Accept	Shows the media format the client can accept
Accept-charset	Shows the character set the client can handle
Accept-encoding	Shows the encoding scheme the client can handle
Accept-language	Shows the language the client can accept
Authorization	Shows what permissions the client has
Host	Shows the host and port number of the client
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Cookie	Returns the cookie to the server (explained later)
If-Modified-Since	If the file is modified since a specific date

Table 26.3 *Response header names*

<i>Header</i>	<i>Description</i>
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Server	Gives information about the server
Set-Cookie	The server asks the client to save a cookie
Content-Encoding	Specifies the encoding scheme
Content-Language	Specifies the language
Content-Length	Shows the length of the document
Content-Type	Specifies the media type
Location	To ask the client to send the request to another site
Accept-Ranges	The server will accept the requested byte-ranges
Last-modified	Gives the date and time of the last change

Example 26.5

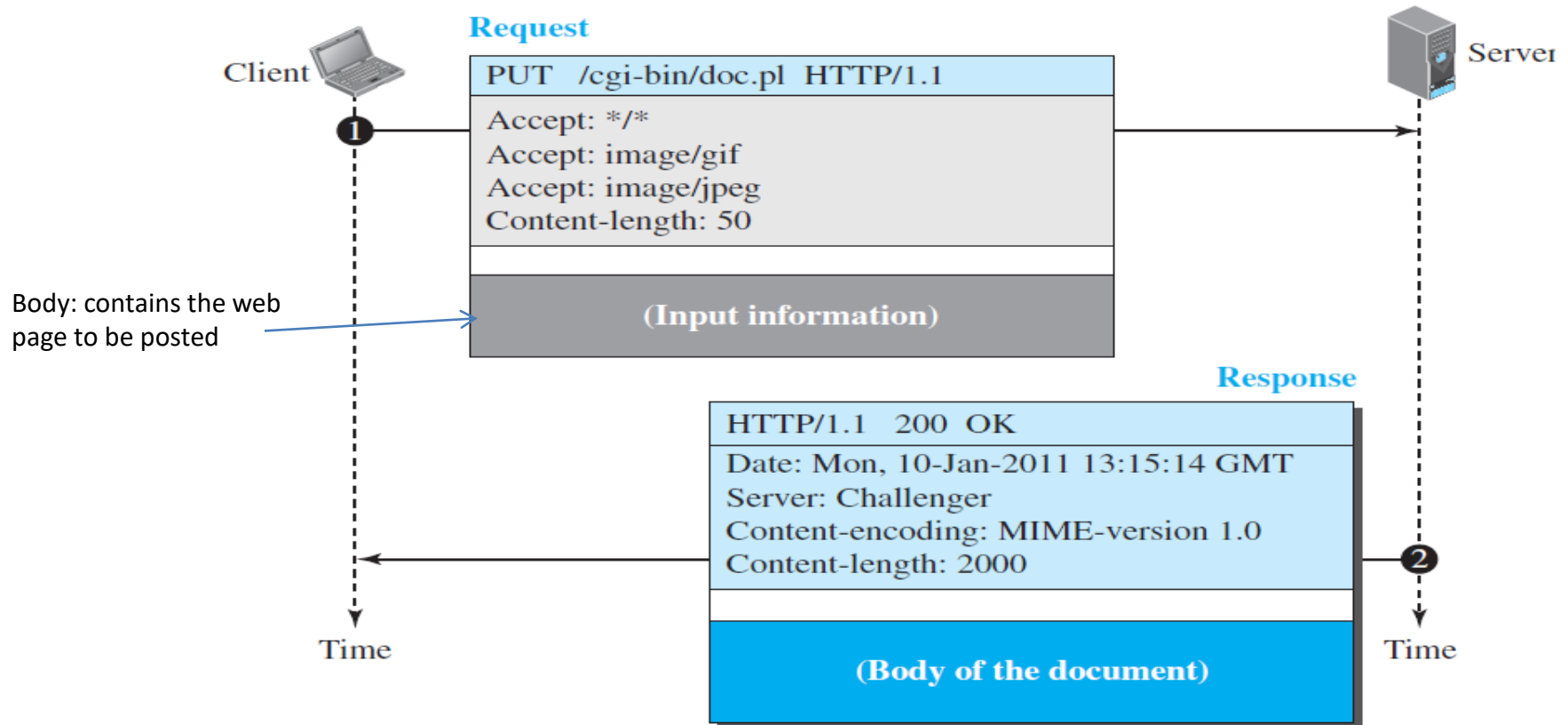
This example retrieves a document (see Figure 26.6). We use the GET method to retrieve an image with the path */usr/bin/image1*



Example 26.6

In this example, the client wants to send a web page to be posted on the server. We use the PUT method.

Figure 26.7 *Example 26.6*



Conditional Request: A client can add a condition in its request.

- One of the most common conditions imposed by the client is the time and date the web page is modified. The client can send the header line *If-Modified-Since* with the request to tell the server that it needs the page only if it is modified after a certain point in time.

Example 26.7

The following shows how a client imposes the modification data and time condition on a request.

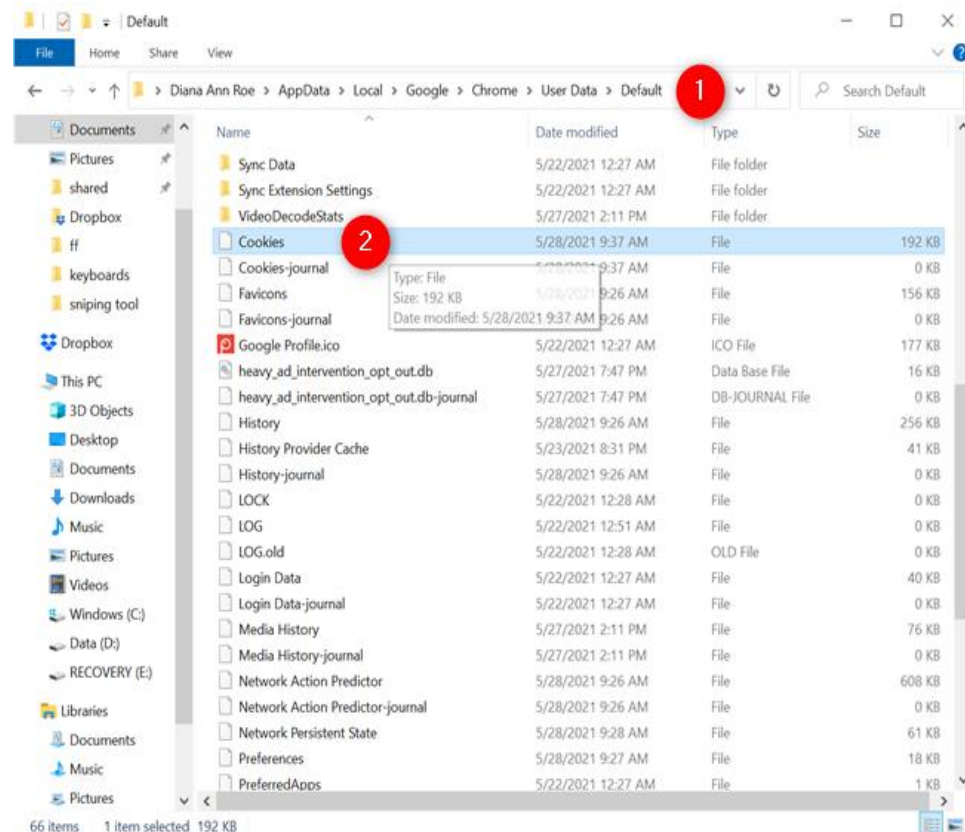
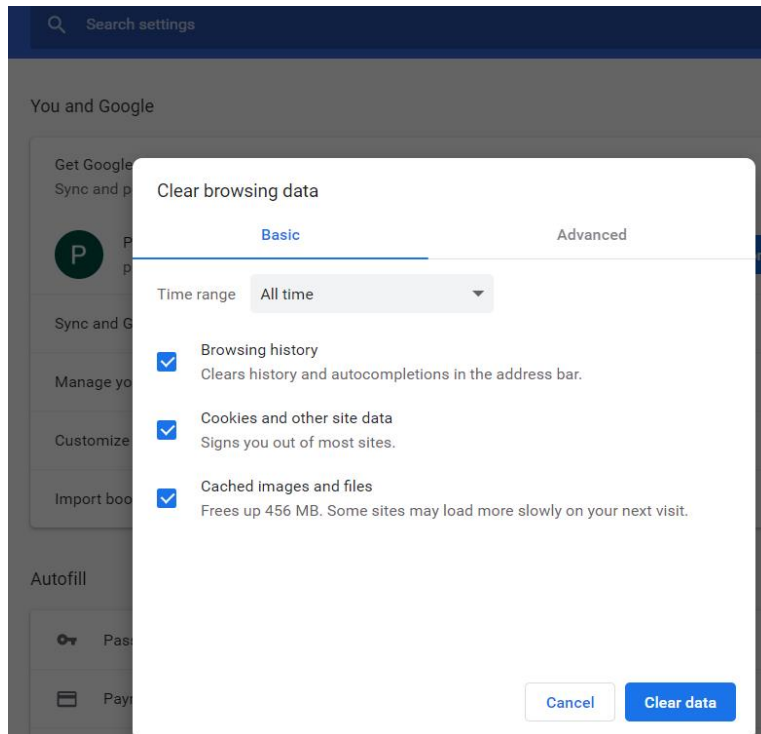
GET http://www.commonServer.com/information/file1 HTTP/1.1	Request line
If-Modified-Since: Thu, Sept 04 00:00:00 GMT	Header line
	Blank line

The status line in the response shows the file was not modified after the defined point in time. The body of the response message is also empty.

HTTP/1.1 304 Not Modified	Status line
Date: Sat, Sept 06 08 16:22:46 GMT	First header line
Server: commonServer.com	Second header line
	Blank line
(Empty Body)	Empty body

Cookies

- **Cookies** are files created by sites you visit. They make your online experience easier by saving browsing information. With **cookies**, sites can: Keep you signed in. Remember your site preferences



Creating and Storing Cookies

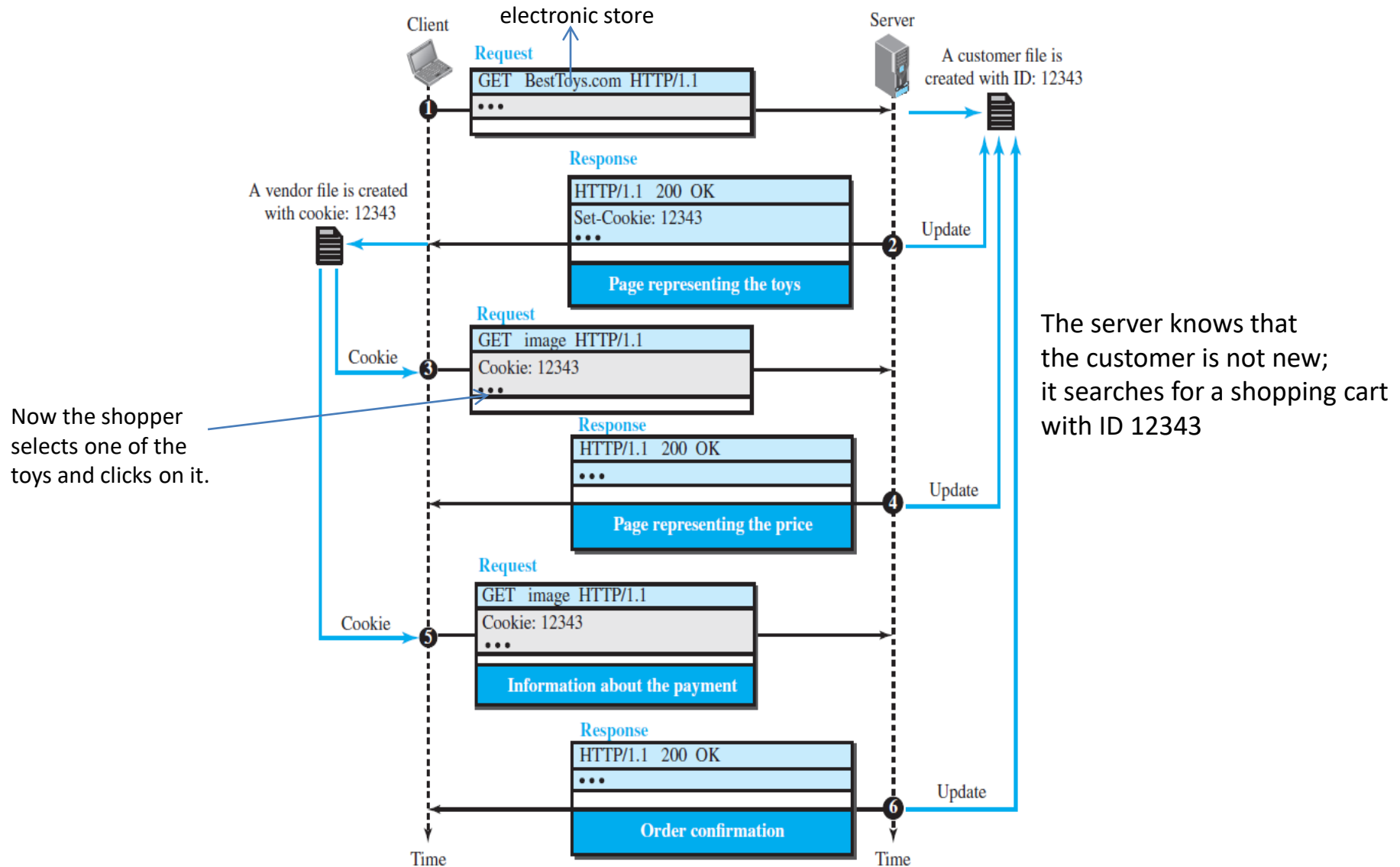
1. When a server receives a request from a client, it stores information about the client in a file or a string. The information may include the domain name of the client, the contents of the cookie (information the server has gathered about the client such as name, registration number, and so on), a timestamp, and other information depending on the implementation.
2. The server includes the cookie in the response that it sends to the client.
3. When the client receives the response, the browser stores the cookie in the cookie directory, which is sorted by the server domain name.

It is a cookie made by the server and eaten by the server

Using Cookies

- A cookie is also used by *advertising* agencies.
- When a user selects her favorite pages, a cookie is made and sent. If the site is accessed again, the cookie is sent to the server to show what the client is looking for.
- An *electronic store* (e-commerce) can use a cookie for its client shoppers. When a client selects an item and inserts it in a cart, a cookie that contains information about the item, such as its number and unit price, is sent to the browser. If the client selects a second item, the cookie is updated with the new selection information, and so on. When the client finishes shopping and wants to check out, the last cookie is retrieved and the total charge is calculated.

A scenario in which an electronic store can benefit from the use of cookies



If the shopper accesses the store sometime in the future, the client sends the cookie again; the store retrieves the file and has all the information about the client.

Web Caching: Proxy Servers

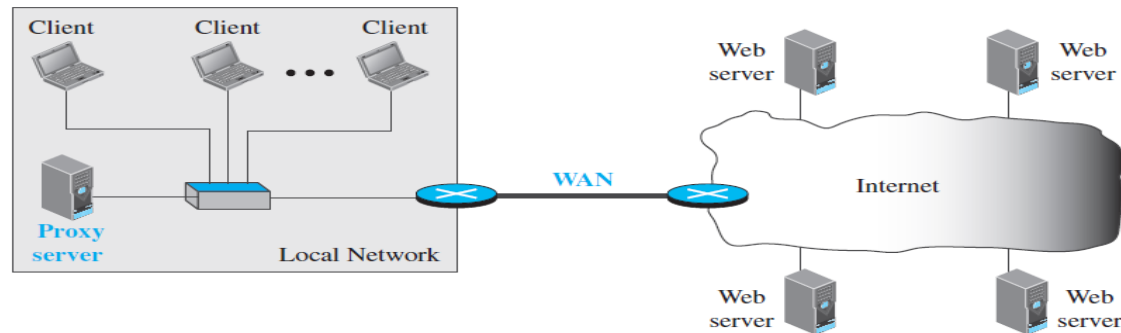
- HTTP supports **proxy servers**.
- A proxy server is a computer that keeps copies of responses to recent requests.
- The HTTP client **sends a request to the proxy server**. The **proxy server checks its cache**. If the response is **not stored** in the cache, the proxy server sends the **request to the corresponding server**.
- The proxy server reduces the load on the original server, decreases traffic, and improves latency
- To use the proxy server, the client must be configured to access the proxy instead of the target server

Proxy Server Location

The proxy servers are normally located at the client site.

1. A client computer can also be used as a proxy server, in a small capacity, that stores responses to requests often invoked by the client.
2. In a company, a proxy server may be installed on the computer LAN to reduce the load going out of and coming into the LAN.
3. An ISP with many customers can install a proxy server to reduce the load going out of and coming into the ISP network.

Figure 26.9 *Example of a proxy server*



When an HTTP request is created by any of the clients (browsers), the request is first directed to the proxy server. If the proxy server already has the corresponding web page, it sends the response to the client. Otherwise, the proxy server acts as a client and sends the request to the web server in the Internet. When the response is returned, the proxy server makes a copy and stores it in its cache before sending it to the requesting client.

Self Study

- ELECTRONIC MAIL: Architecture, Web-Based Mail, E-Mail Security.

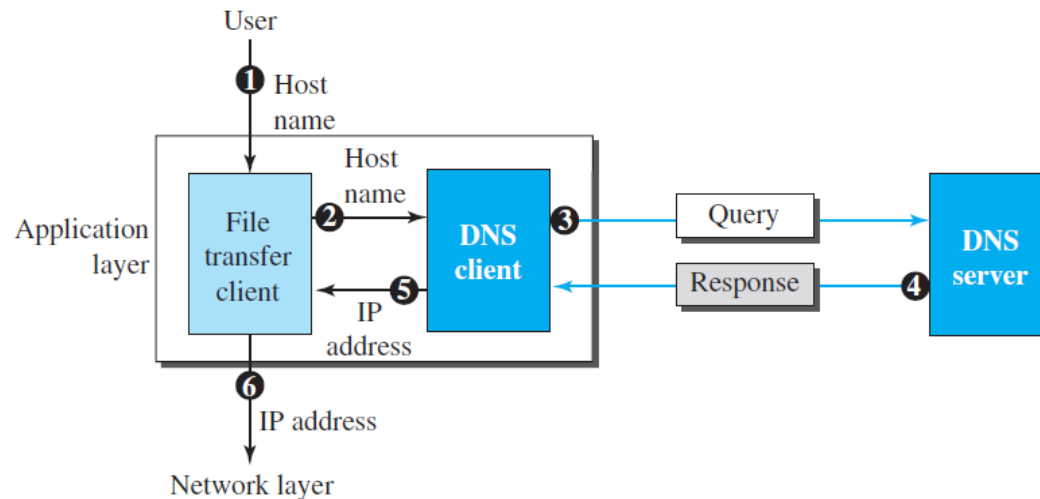
<https://www.youtube.com/watch?v=JkEYOt08-rU>

DNS

- DNS is short for **Domain Name Service** or **Domain Name System**.
- It is an application layer protocol.
- Purpose: DNS is a host name to IP Address translation service



Figure 26.28 *Purpose of DNS*



DNS

Purpose:

The following six steps map the host name to an IP address:

1. The user passes the host name to the file transfer client.
2. The file transfer client passes the host name to the DNS client.
3. Each computer, after being booted, knows the address of one DNS server. The DNS client sends a message to a DNS server with a query that gives the file transfer server name using the known IP address of the DNS server.
4. The DNS server responds with the IP address of the desired file transfer server.
5. The DNS server passes the IP address to the file transfer client.
6. The file transfer client now uses the received IP address to access the file transfer server.

Name Space

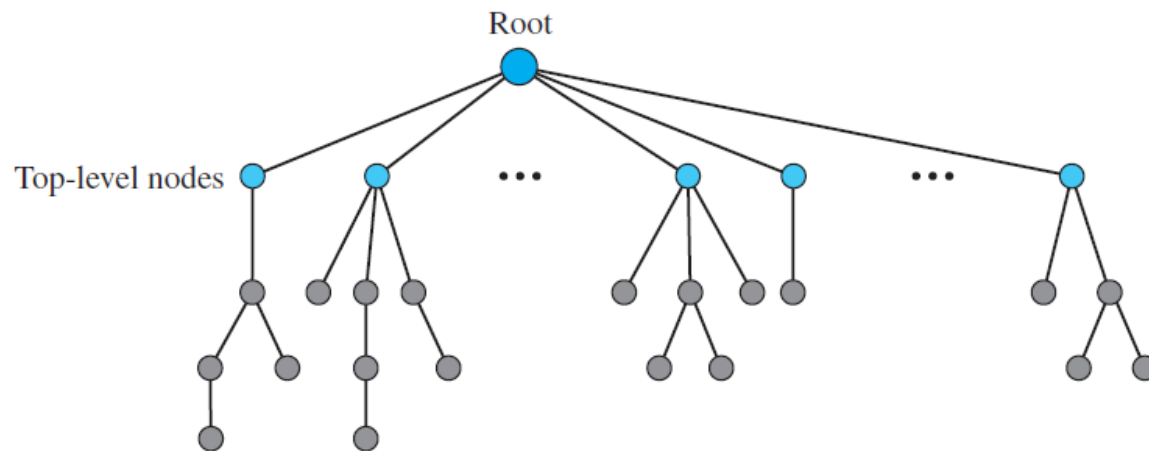
A **name space** that maps each address to a unique name can be organized in ways: flat or hierarchical.

- In a *flat name space*, a name is assigned to an address. The main disadvantage of a flat name space is that it cannot be used in a large system such as the Internet because it must be centrally controlled to avoid ambiguity and duplication.
- In a *hierarchical name space*, each name is made of several parts. The first part can define the nature of the organization, the second part can define the name of an organization, the third part can define departments in the organization, and so on.

Domain Name Space

- To have a hierarchical name space, a **domain name space** was designed. In this design the names are defined in an inverted-tree structure with the root at the top. The tree can have only 128 levels: level 0 (root) to level 127

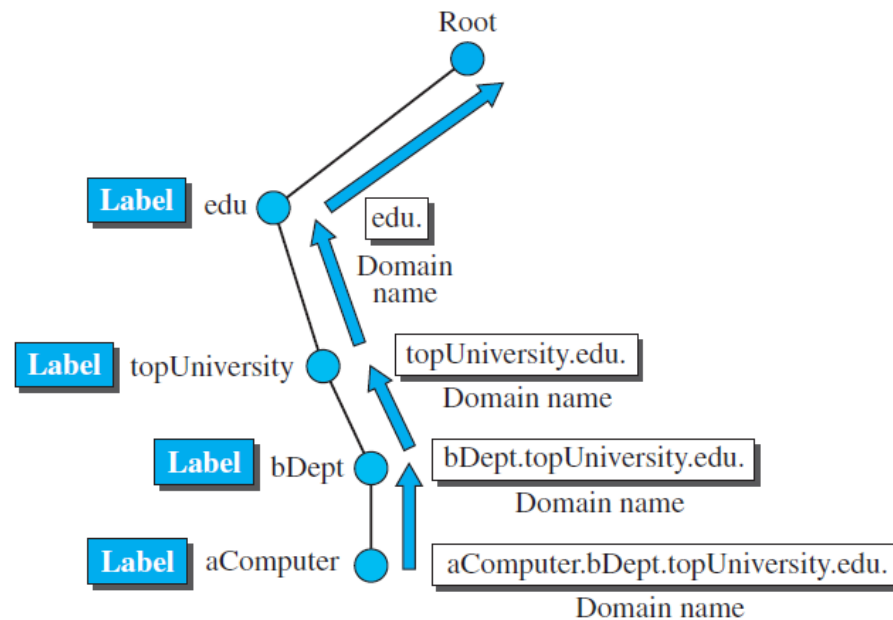
figure 26.29 *Domain name space*



Label

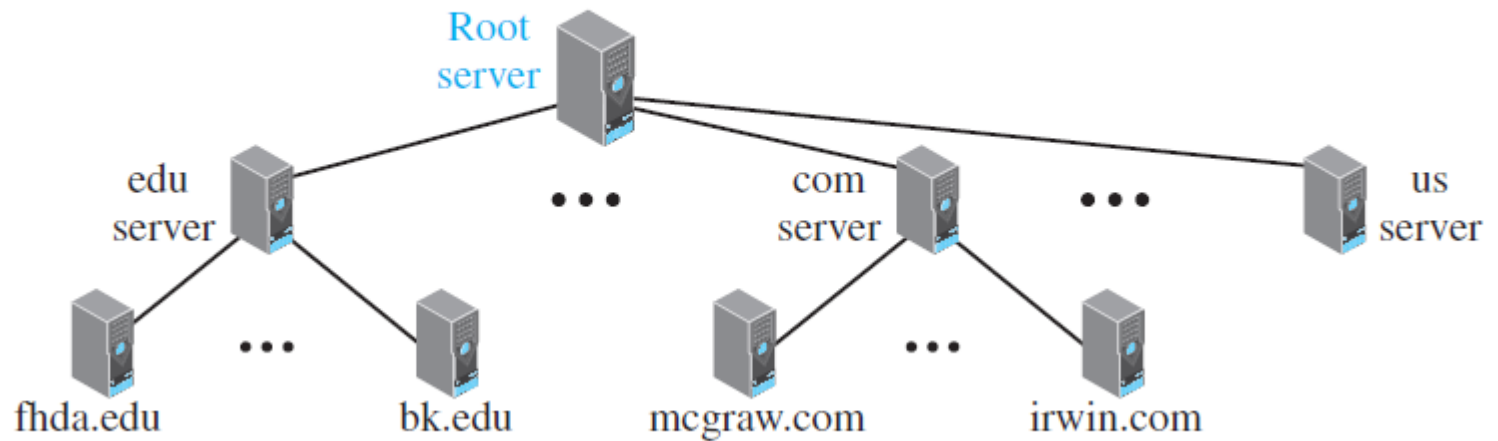
- Each node in the tree has a **label**, which is a string with a maximum of 63 characters. The root label is a null string (empty string).

Figure 26.30 *Domain names and labels*



Hierarchy of Name Servers

Figure 26.32 *Hierarchy of name servers*

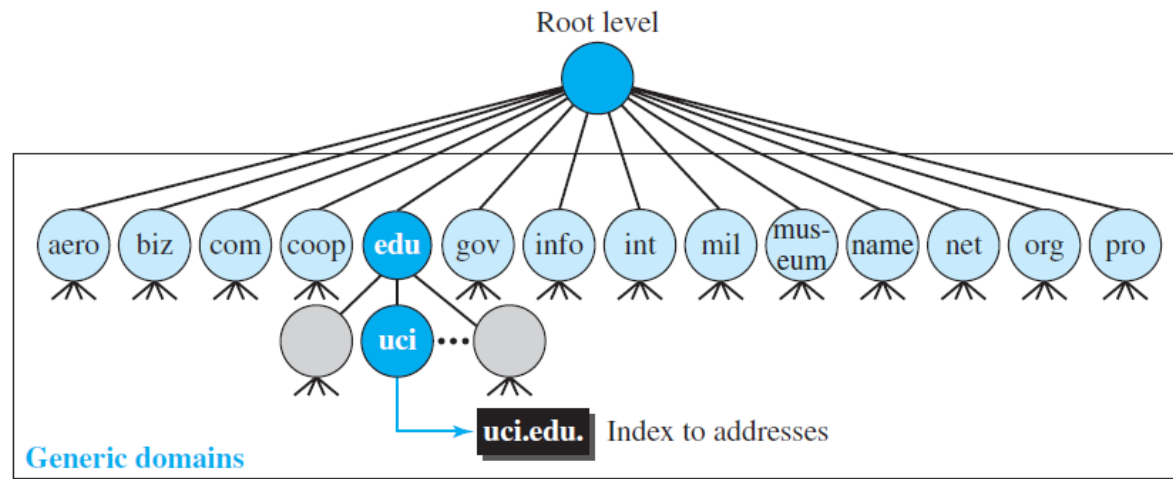


DNS in the Internet

In the Internet, the domain name space (tree) was originally divided into three different sections: generic domains, country domains, and the inverse domains. However, due to the rapid growth of the Internet, it became extremely difficult to keep track of the inverse domains. We, therefore, concentrate on the first two.

Generic Domains: The **generic domains** define registered hosts according to their generic behavior.

Figure 26.34 *Generic domains*



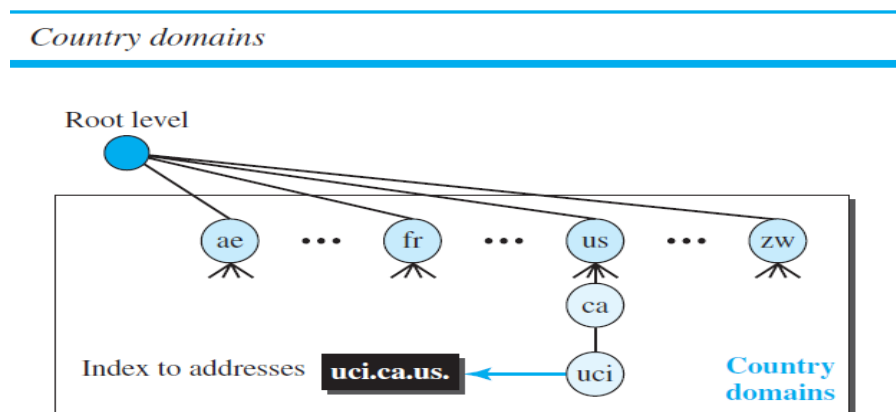
DNS

- Looking at the tree, we see that the first level in the **generic domains** section allows 14 possible labels.

Table 26.12 *Generic domain labels*

<i>Label</i>	<i>Description</i>	<i>Label</i>	<i>Description</i>
aero	Airlines and aerospace	int	International organizations
biz	Businesses or firms	mil	Military groups
com	Commercial organizations	museum	Museums
coop	Cooperative organizations	name	Personal names (individuals)
edu	Educational institutions	net	Network support centers
gov	Government institutions	org	Nonprofit organizations
info	Information service providers	pro	Professional organizations

- Country Domains: The **country domains** section uses two-character country abbreviations (e.g., us for United States).

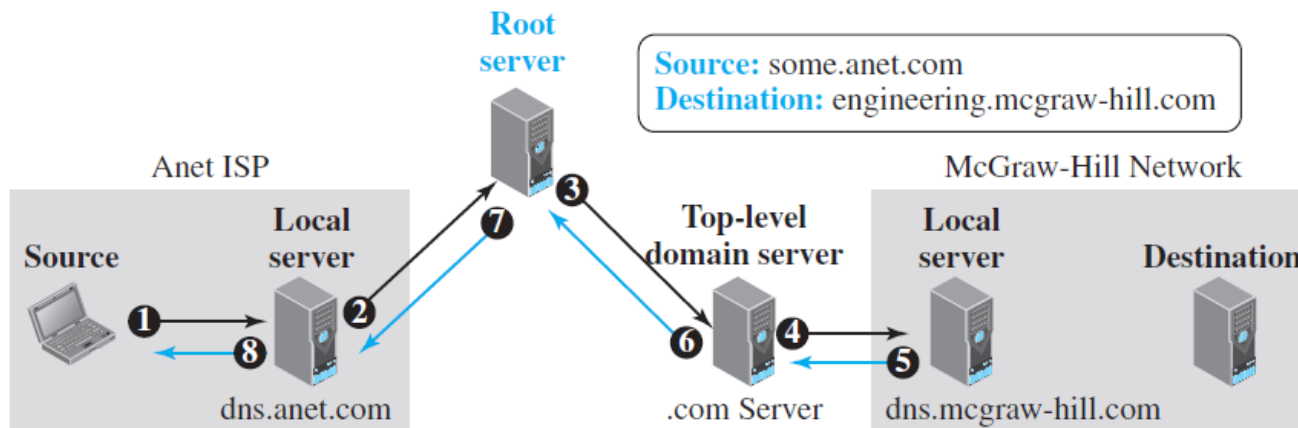


Resolution

- Mapping a name to an address is called *name-address resolution*
- A host that needs to map an address to a name or a name to an address calls a DNS client called a *resolver*
- The resolver accesses the closest DNS server with a mapping request. If the server has the information, it satisfies the resolver; otherwise, it either refers the resolver to other servers or asks other servers to provide the information. After the resolver receives the mapping, it interprets the response to see if it is a real resolution or an error, and finally delivers the result to the process that requested it.
- A resolution can be either **recursive or iterative**.

Recursive Resolution

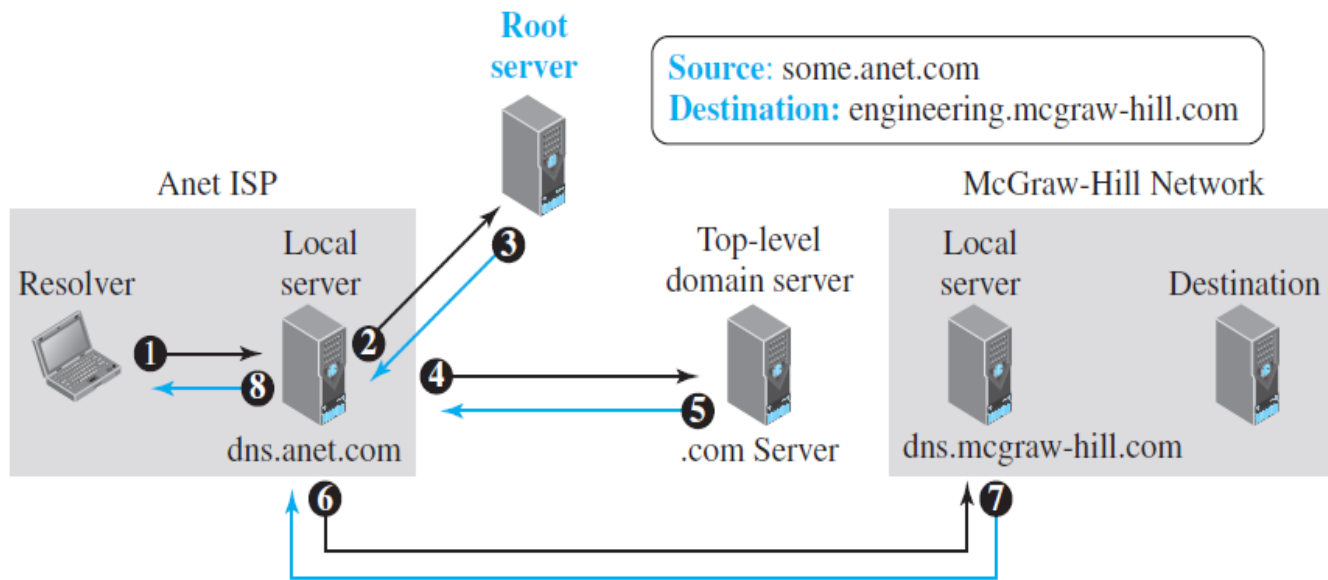
Figure 26.36 Recursive resolution



- Recursive query is between a DNS client and its local DNS server
- The Recursive query demands a name resolution or the answer –it can't be found

Iterative Resolution

Figure 26.37 *Iterative resolution*



- Iterative query is between a local DNS server and other DNS servers.
- The Iterative query do not demand a name resolution, which means other DNS servers may provide a name resolution if they know or simply respond with a referral.

QUALITY OF SERVICE

Quality of service (QoS) is an internetworking issue that refers to a set of techniques and mechanisms **that guarantee the performance of the network** to deliver predictable service to an application program.

-Mechanisms or technologies **to control traffic and ensure the performance of the network**

DATA-FLOW CHARACTERISTICS: *reliability, delay, jitter, and bandwidth*

Reliability: a characteristic that a flow needs in order to deliver the packets safe and sound to the destination

For example, reliable transmission is more important for electronic mail, file transfer, and Internet access than for telephony or audio conferencing

Delay: applications can tolerate delay in different degrees.

Ex: Telephony, audio conferencing, video conferencing, and remote logging need minimum delay

Jitter: Jitter is the variation in delay for packets belonging to the same flow

Bandwidth: Different applications need different bandwidths

Sensitivity of Applications

Table 30.1 *Sensitivity of applications to flow characteristics*

<i>Application</i>	<i>Reliability</i>	<i>Delay</i>	<i>Jitter</i>	<i>Bandwidth</i>
FTP	High	Low	Low	Medium
HTTP	High	Medium	Low	Medium
Audio-on-demand	Low	Low	High	Medium
Video-on-demand	Low	Low	High	High
Voice over IP	Low	High	High	Low
Video over IP	Low	High	High	High

Flow Classes

Example 30.1

Although the Internet has not defined flow classes formally, the ATM protocol does. As per ATM specifications, there are five classes of defined service.

- a. ***Constant Bit Rate (CBR)***. This class is used for emulating circuit switching. CBR applications are quite sensitive to cell-delay variation. Examples of CBR are telephone traffic, video conferencing, and television.
- b. ***Variable Bit Rate-Non Real Time (VBR-NRT)***. Users in this class can send traffic at a rate that varies with time depending on the availability of user information. An example is multimedia e-mail.
- c. ***Variable Bit Rate-Real Time (VBR-RT)***. This class is similar to VBR-NRT but is designed for applications such as interactive compressed video that are sensitive to cell-delay variation.
- d. ***Available Bit Rate (ABR)***. This class of ATM services provides rate-based flow control and is aimed at data traffic such as file transfer and e-mail.
- e. ***Unspecified Bit Rate (UBR)***. This class includes all other classes and is widely used today for TCP/IP.

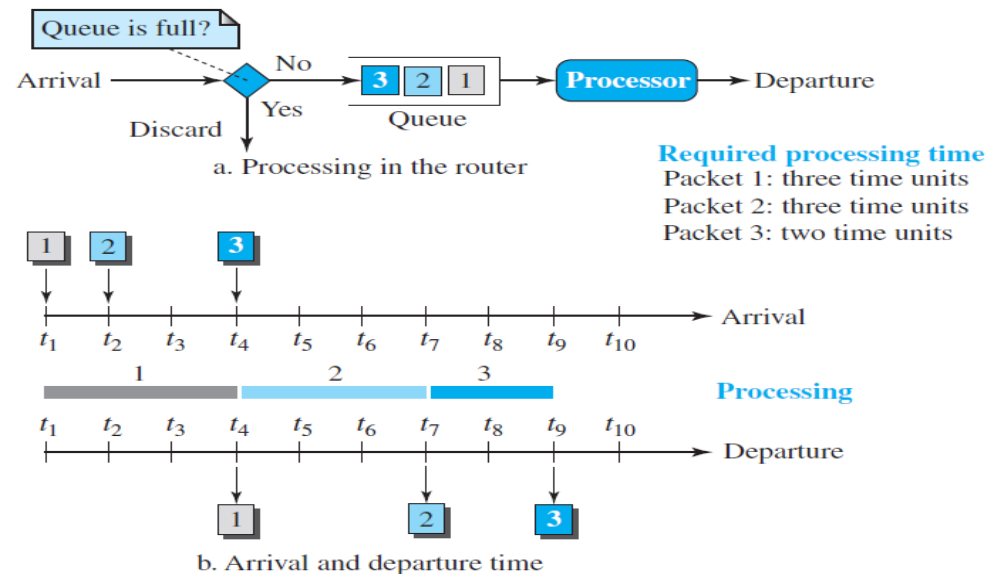
FLOW CONTROL TO IMPROVE QOS

- **Scheduling:** consists of a list of times at which possible tasks, events, or actions are intended to take place - Routers
 - *FIFO Queuing, Priority Queuing, Weighted Fair Queuing*
- **Traffic Shaping or Policing**
 - *Leaky Bucket, Token Bucket*
- **Resource Reservation**
- **Admission Control**

FIFO Queuing

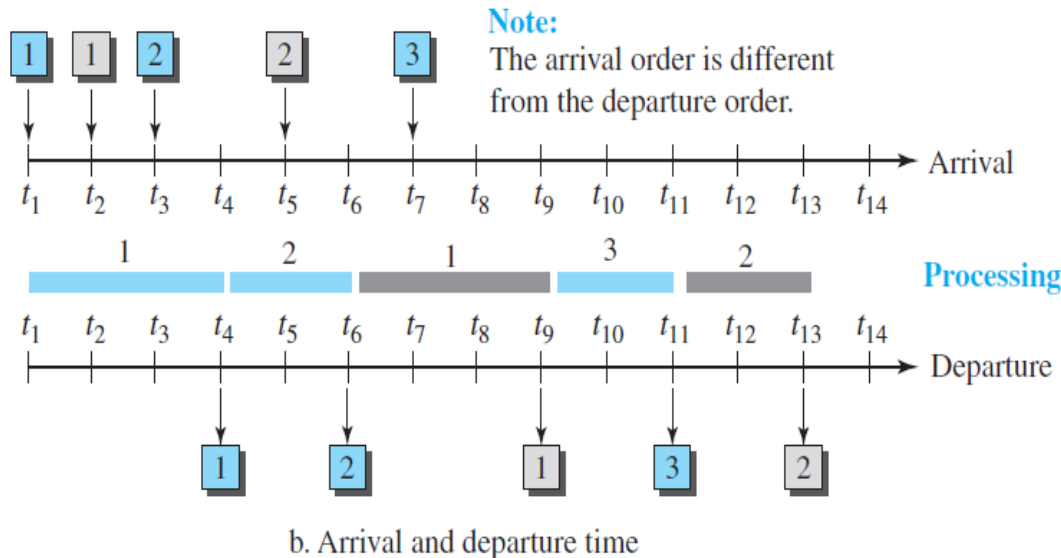
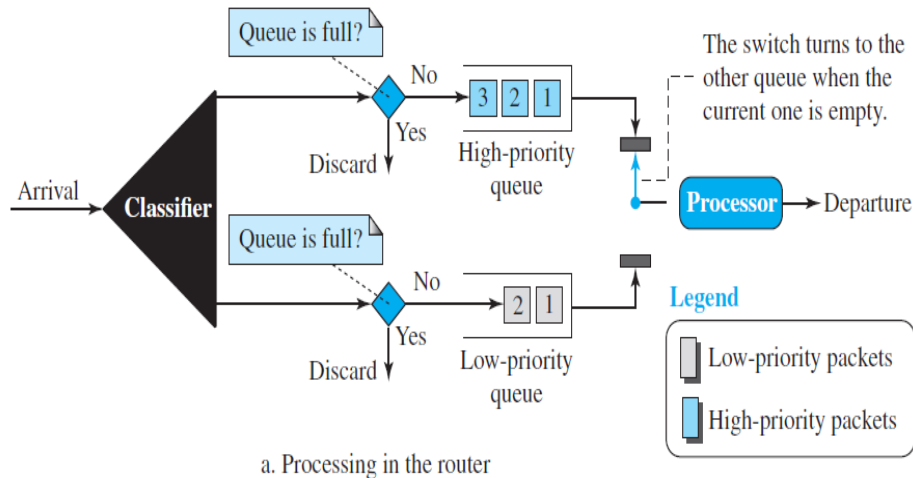
- The packets depart in the order they arrive
- Packets wait in a buffer (queue) until the node (router) is ready to process them. Packets from different applications (with different sizes) arrive at the queue, are processed, and depart.
- In the figure, packets 1 and 2 need three time units of processing (large), but packet 3, which is smaller, needs two time units.
- With FIFO queuing, all packets are treated the same in a packet-switched network (FTP, or Voice over IP, or an e-mail message-equally subject to loss, delay, and jitter)

Figure 30.1 FIFO queue



Priority Queuing

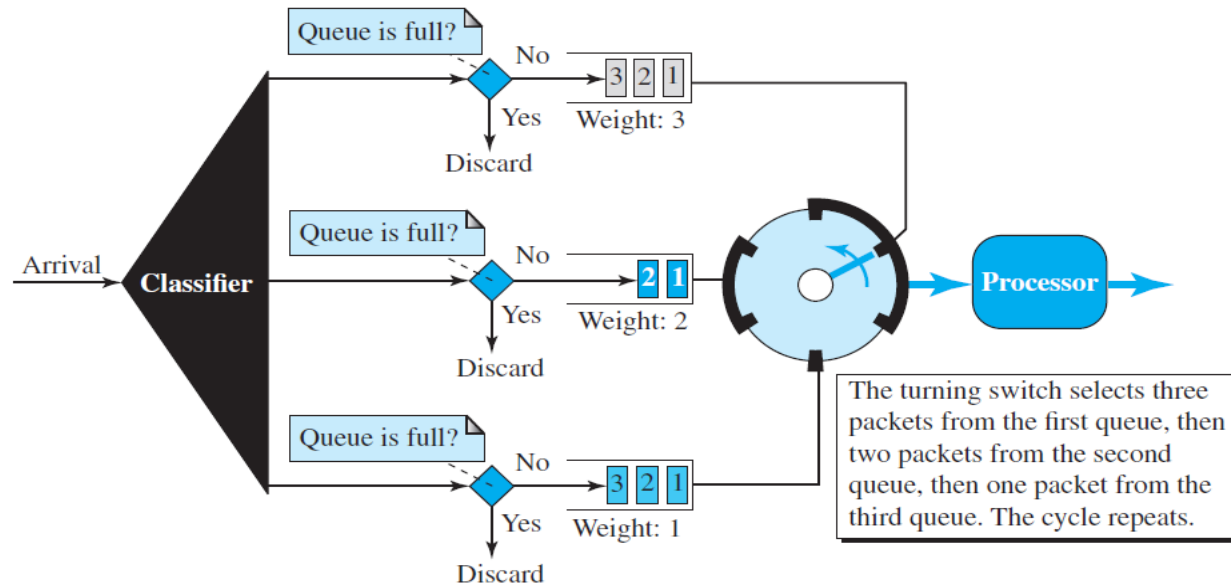
Figure 30.2 Priority queuing



- In **priority queuing**, packets are first assigned to a priority class
- The packets in the highest-priority queue are processed first. Packets in the lowest-priority queue are processed last
- A packet priority is determined from a specific field in the packet header: the **ToS field of an IPv4 header**, the **priority field of IPv6 header**
- Advantage: priority queue can provide better QoS than the FIFO queue because higher-priority traffic, such as multimedia, can reach the destination with less delay
- If there is a continuous flow in a high-priority queue, the packets in the lower-priority queues will never have a chance to be processed. This is a condition called *starvation*

Weighted Fair Queuing

Figure 30.3 Weighted fair queuing



- In this technique, the packets are still assigned to different classes and admitted to different queues.
- The queues, however, are weighted based on the priority of the queues; higher priority means a higher weight.
- The system processes packets in each queue in a round-robin fashion-based on the corresponding weight

Resource Reservation

- A flow of data needs resources such as a buffer, bandwidth, CPU time, and so on.
- The quality of service is improved if these resources are reserved beforehand.

Admission Control

- Admission control refers to the mechanism used by a router or a switch to accept or reject a flow based on predefined parameters called *flow specifications*
- It takes into account bandwidth, buffer size, CPU speed, etc., as well as its previous commitments to other flows

Traffic Shaping or Policing

- To control the amount and the rate of traffic is called *traffic shaping* or *traffic policing*
- *traffic shaping*: when the traffic leaves a network
- *traffic policing*: when the data enters the network
- Two techniques can shape or police the traffic: leaky bucket and token bucket.

Leaky Bucket

- The input rate can vary, but the output rate remains constant.
- **leaky bucket** can smooth out bursty traffic.

Figure 30.4 Leaky bucket

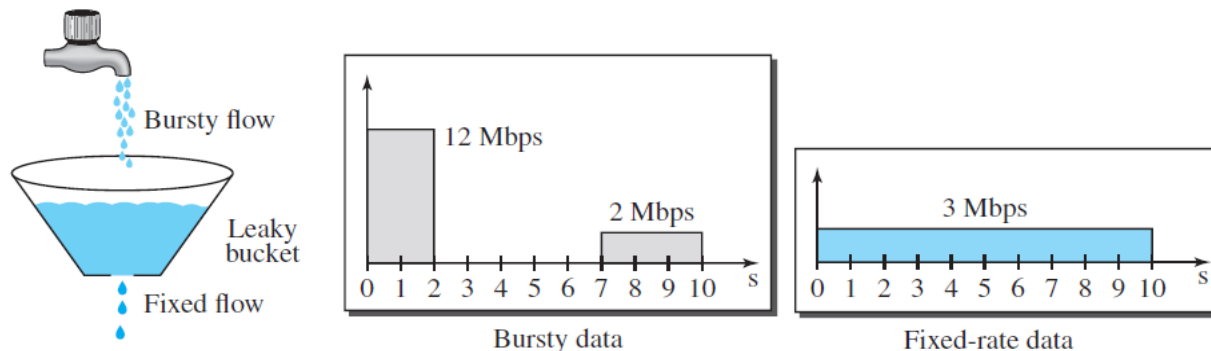
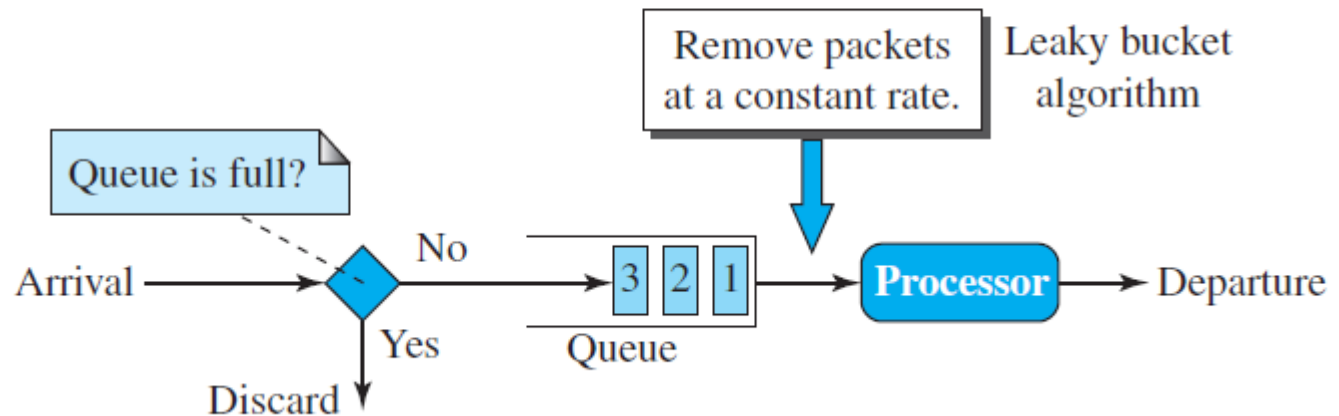


Figure 30.5 *Leaky bucket implementation*



A leaky bucket algorithm shapes bursty traffic into fixed-rate traffic by averaging the data rate. It may drop the packets if the bucket is full.

Token Bucket

The token bucket allows bursty traffic at a regulated maximum rate

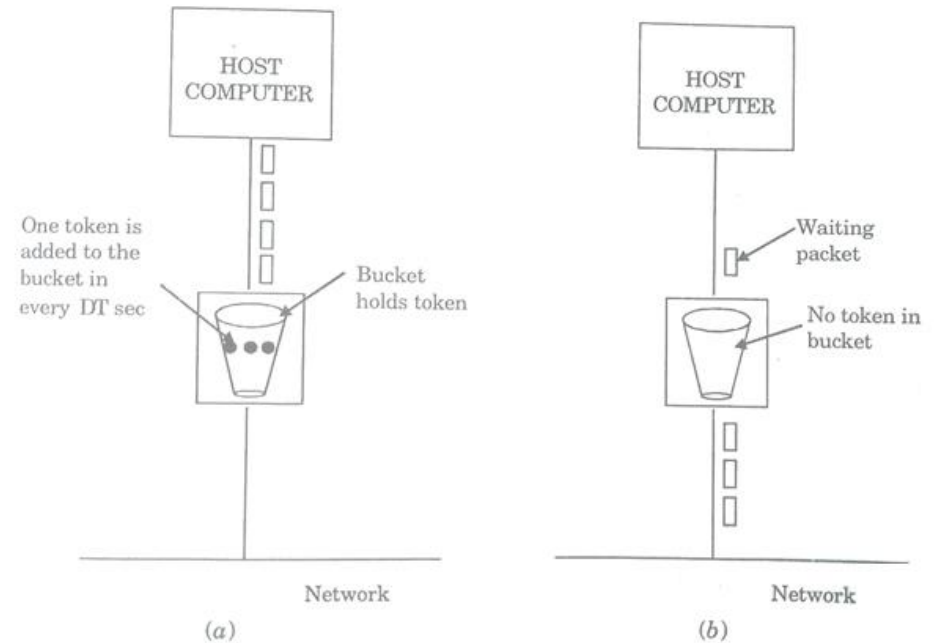
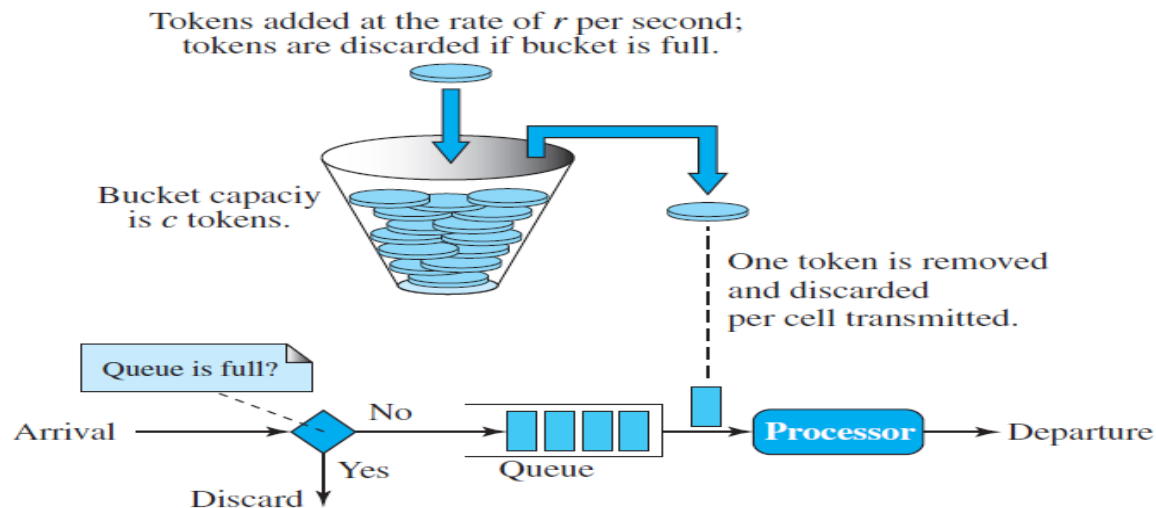


Fig: Token Bucket System (a) Before tokens are captured (b) After tokens are captured

Figure 30.6 Token bucket



Token Bucket

- The capacity of the bucket is c tokens and tokens enter the bucket at the rate of r tokens per second.

$$\text{Maximum number of packets} = r \times t + c$$

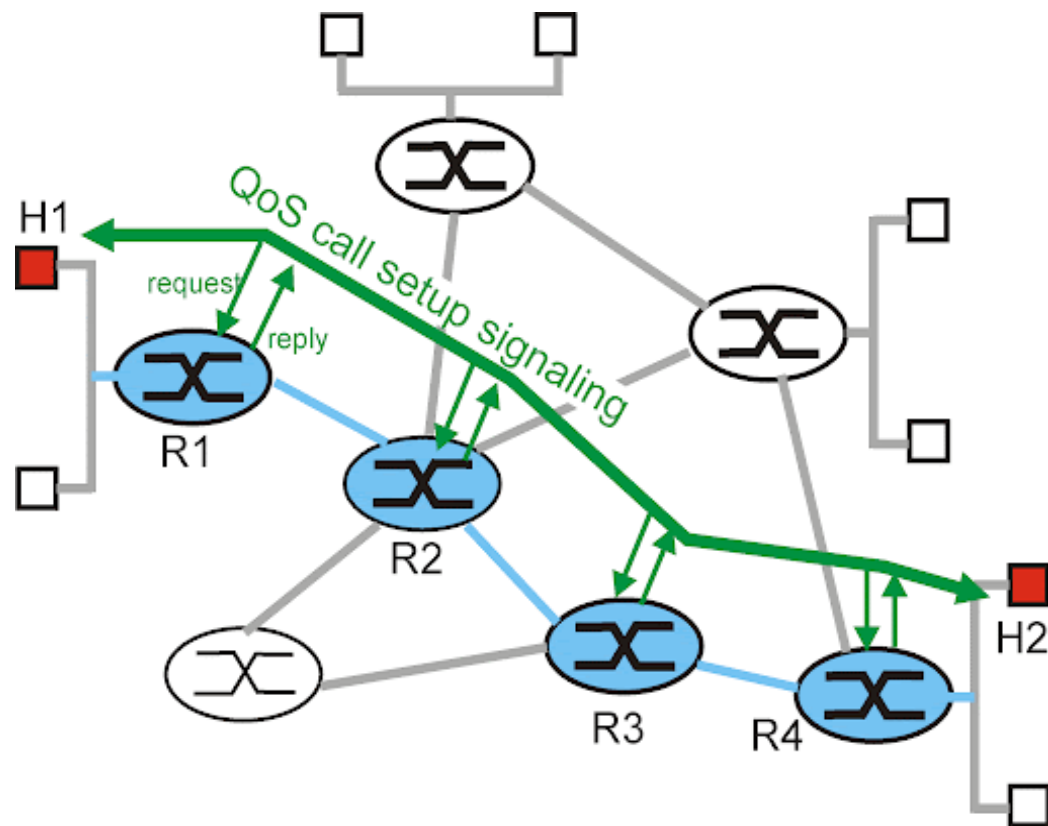
The maximum average rate for the token bucket is shown below.

$$\text{Maximum average rate} = (r \times t + c)/t \text{ packets per second}$$

Parameter	Leaky Bucket	Token Bucket
Tokens	Token Independent	Token Dependent
Overflow	If Bucket overflows Packets are discarded	If Bucket overflows Tokens are discarded not Packets
DATA Flow rate	Constant	Variable
Support for Bursty traffic	Not Supported	Supported if sufficient tokens are available
When Application is idle	Does Nothing	Generates the tokens to support bursty traffic
Packet transfer	Packets transfer continuously with constant rate	Packets can be transferred if and only if tokens are available

INTEGRATED SERVICES (INTSERV)

- In this model, which is a *flow-based* architecture, resources such as bandwidth are explicitly reserved for a given data flow. To provide different QoS for different applications, IETF developed the **Integrated Services (IntServ)** model.
- Integrated Services is flow-based, which means that all accommodations need to be made before a flow can start.
 - we need a connection-oriented service at the network layer
 - A connection establishment phase is needed to inform all routers of the requirement and get their approval (admission control).
 - since IP is currently a connectionless protocol, we need another protocol to be run on top of IP to make it a connection-oriented protocol. This protocol is called Resource Reservation Protocol (RSVP)

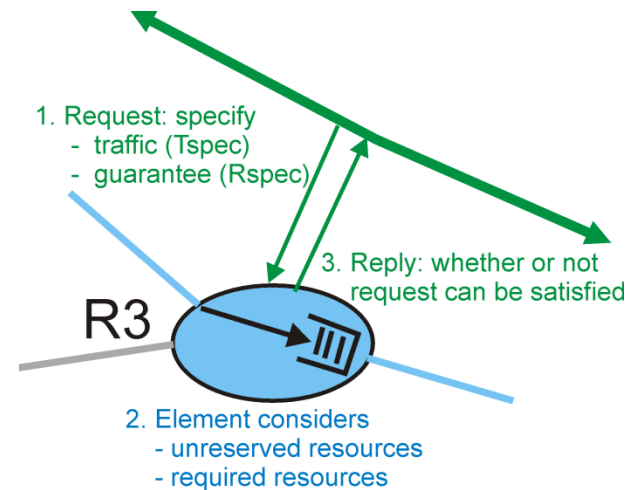


• Flow Specification

To define a specific flow, a source needs to define a flow specification, which is made of two parts:

1. *Rspec (resource specification)*. Rspec defines the resource that the flow needs to reserve (buffer, bandwidth, etc.).

2. *Tspec (traffic specification)*. Tspec defines the traffic characterization of the flow



• Admission

- After a router receives the flow specification from an application, it decides to admit or deny the service.
- The decision is based on the previous commitments of the router and the current availability of the resource.

- **Service Classes:** Two classes of services have been defined for Integrated Services: guaranteed service and controlled-load service.

Guaranteed Service Class:

- This type of service is designed for *real-time traffic* that needs a guaranteed minimum end-to-end delay
- This type of service guarantees that the packets will arrive within a certain delivery time and are not discarded if flow traffic stays within the boundary of T_{spec}
- guaranteed services are *quantitative services*, in which the amount of end-to-end delay and the data rate must be defined by the application
- Normally guaranteed services are required for real-time applications (voice over IP).

Controlled-Load Service Class

- This type of service is designed for applications that can accept some delays but are sensitive to an overloaded network and to the danger of losing packets
- Good examples of these types of applications are file transfer, e-mail, and Internet access
- The controlled load service is a *qualitative service* in that the application requests the possibility of low-loss or no-loss packets.

Resource Reservation Protocol (RSVP)

- RSVP to provide resource reservations for all kinds of traffic
- Based on multicast communication

Receiver-Based Reservation

- In RSVP, the receivers make the reservation. This strategy matches the other multicasting protocols.
- For example, in multicast routing protocols, the receivers, not the sender, make a decision to join or leave a multicast group.

RSVP Messages

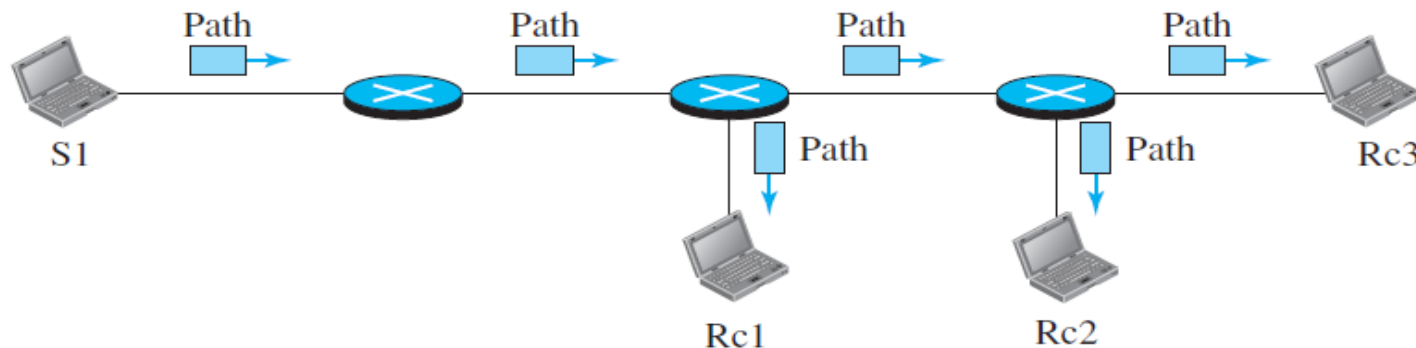
- *Path* and *Resv*

RSVP Messages

Path Messages

- The path is needed for the reservation.
- Path message travels from the sender and reaches all receivers in the multicast path.
- On the way, a Path message stores the necessary information for the receivers. A Path message is sent in a multicast environment; a new message is created when the path diverges.

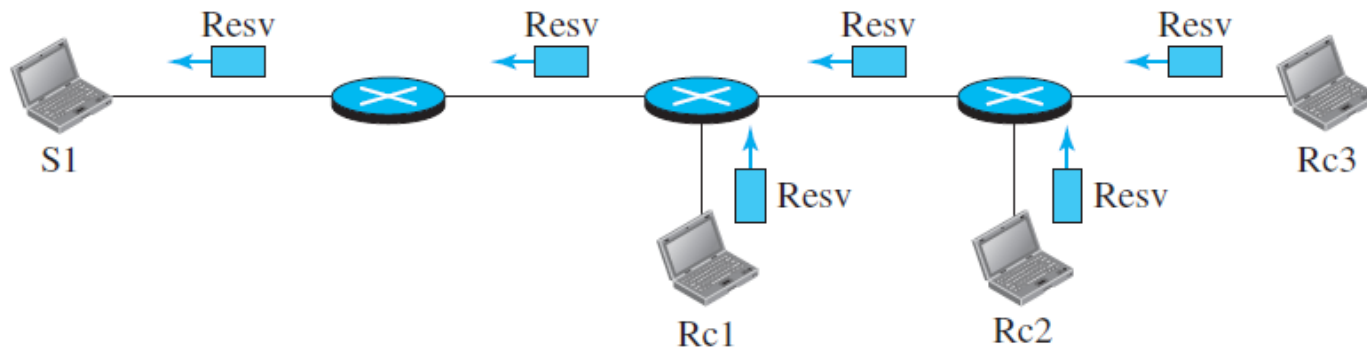
Figure 30.7 *Path messages*



Resv Messages

- After a receiver has received a Path message, it sends a Resv message.
- The Resv message travels toward the sender (upstream) and makes a resource reservation on the routers that support RSVP.
- If a router on the path does not support RSVP, it routes the packet based on the best-effort delivery methods (the network does not provide any guarantee that data is delivered).

Figure 30.8 *Resv messages*



Problems with Integrated Services

- ***Service-Type Limitation*** The Integrated Services model provides only two types of services, guaranteed and control-load
- ***Scalability***
- The Integrated Services model requires that each router keep information for each flow. As the Internet is growing every day, this is a serious problem.
- Keeping information is especially troublesome for core routers because they are primarily designed to switch packets at a high rate and not to process information

DIFFERENTIATED SERVICES (DIFFSERV)

- packets are **marked** by applications into classes according to their **priorities**
- The routers do not have to store information about flows. The applications, or hosts, define the type of service they need each time they send a packet. – avoids scalability problem
- The router routes the packet based on the class of service defined in the packet, not the flow – avoids Service-Type Limitation
- In DiffServ, each packet contains a field called the DS field
- IETF proposes to replace the existing ToS (type of service) field in IPv4 or the priority class field in IPv6 with the DS field

Figure 30.10 *DS field*



- The DS field contains two subfields: DSCP and CU
- The DSCP (Differentiated Services Code Point) is a 6-bit subfield that defines the **per-hop behavior (PHB)**.
- The 2-bit CU (Currently Unused) subfield is not currently used

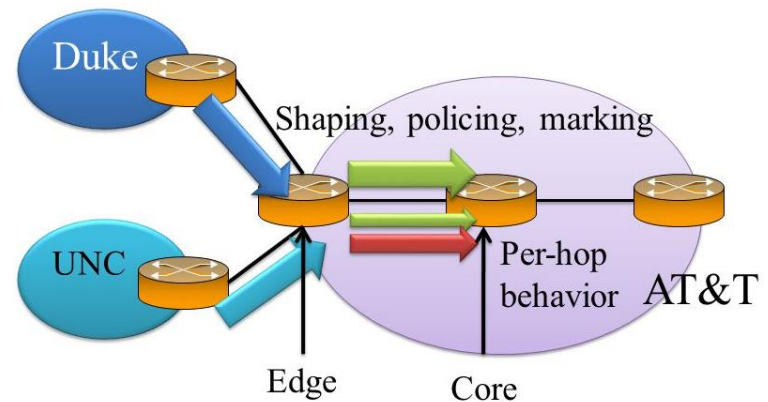
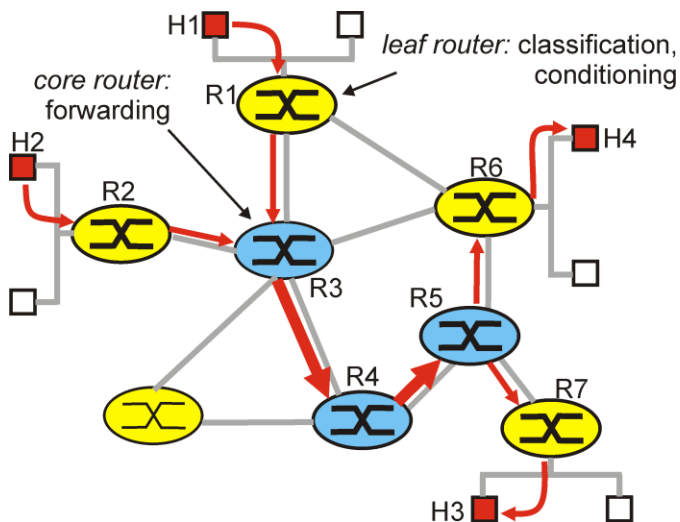
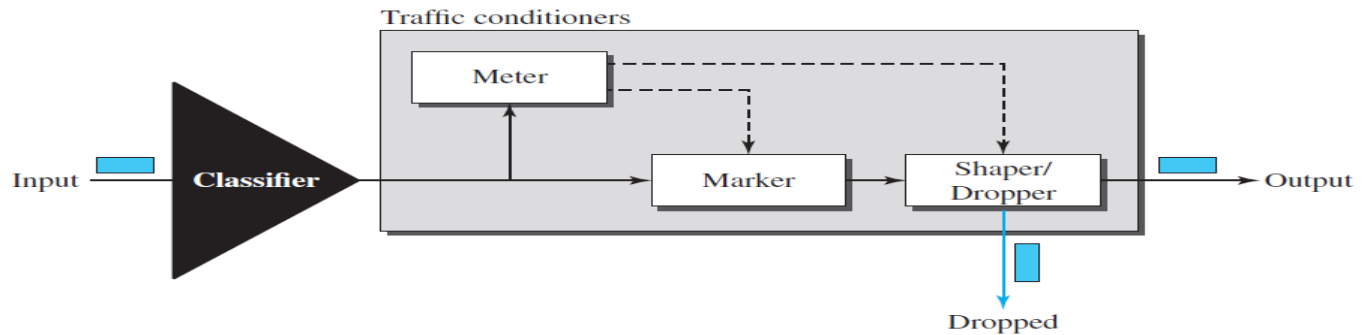
Per-Hop Behavior

- The DiffServ model defines per-hop behaviors (PHBs) for each node that receives a packet. So far three PHBs are defined: DE PHB, EF PHB, and AF PHB
- **DE PHB:** The DE PHB (default PHB) is the same as best-effort delivery, which is compatible with ToS.
- **EF PHB:** provides the following services: **a.** Low loss. **b.** Low latency.
c. Ensured bandwidth.
- **AF PHB:** delivers the packet with a high assurance as long as the class traffic does not exceed the traffic profile of the node.

Traffic Conditioners

- To use quality of service (QoS) policies (DiffServ), network equipment (like routers and switches) must have the capability for traffic conditioners such as meters, markers, shapers, and droppers,

Figure 30.11 *Traffic conditioners*



Traffic Conditioners

- meters, which measure traffic and check it against traffic profiles
- Packet markers set the Differentiated Services field
- shapers, which delay traffic to make it conform to a certain traffic profile
- droppers, which simply drop packets